Lecture 7 : Advanced Edge Detection

1. What is edge detection : A brief summary of the previous lecture.
2. Problems in Edge Detection : **Do template and gradient based operators work ?**
3. Measures of Performance
4. Canny Edge Detection
5. Marr – Hildreth Edge Detection

What is Edge Detection?

- An edge is a boundary between an object and the background, and indicates the boundary between overlapping objects.
- Edges form the outline of an object, so if the edges are identified in an image accurately, then all the objects can be located and basic properties such as area, perimeter can be measured.
- Edge detection is based on the relationship a pixel has with its neighbors.

Edge Detection ...

- Edges, in an image are defined as locations where there is a significant variation in the gray level or color of pixel in some direction.
- Edge detection extracts and localizes points (pixels) around which a large change in image brightness has occurred.
- This can help in the process of *segmentation*

Problems in Edge Detection

- Problems:-

1) Because of digitization:-The image may be sampled in such a way so that change in gray level may extend across some number of pixels.
2) Because of noise:-Due to factors such as light intensity, type of camera and lens, motion, temperature, dust and others.

Do Gradient operators work well?

Gradient operators are approximated as :

Δ_{x2} : $A(x, y) = A(x+1, y) - A(x-1, y)$

Δ_{y2} : $A(x, y) = A(x, y+1) - A(x, y-1)$

This operator is symmetrical with respect to the pixel (x, y). It does not consider the value of the pixel at (x, y).

The edge response is given by:

$$G_{mag} = \sqrt{((\partial A/\partial x)^2 + (\partial A/\partial y)^2)}$$

and the direction of the edge is approximately:

$$G_{dir} = \text{atan}\ (\partial A/\partial y) / (\partial A/\partial x)$$

Edge pixel- which exceeds the threshold value

Gradient operators examples

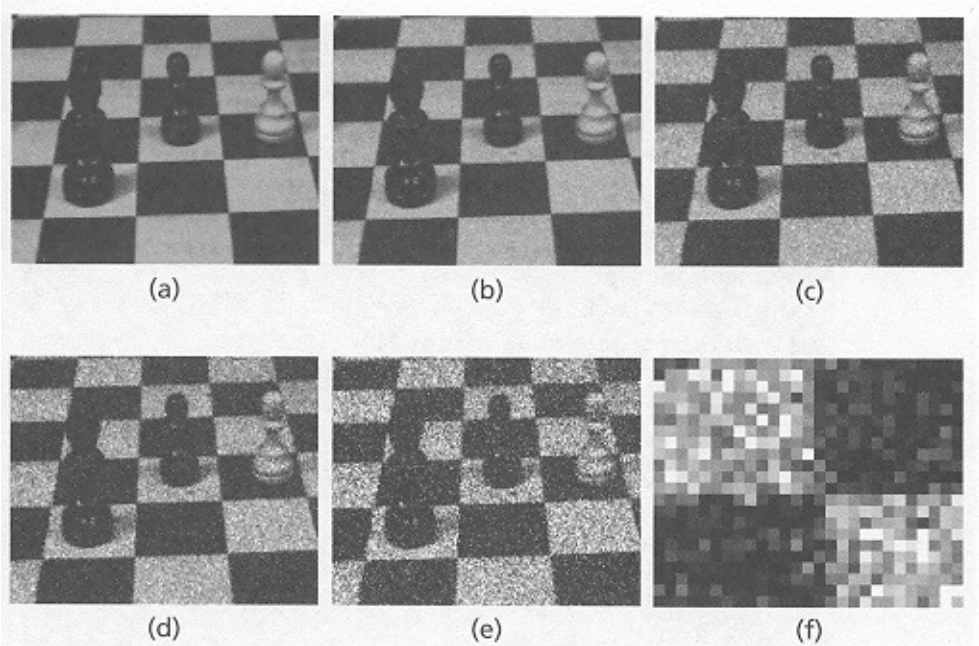
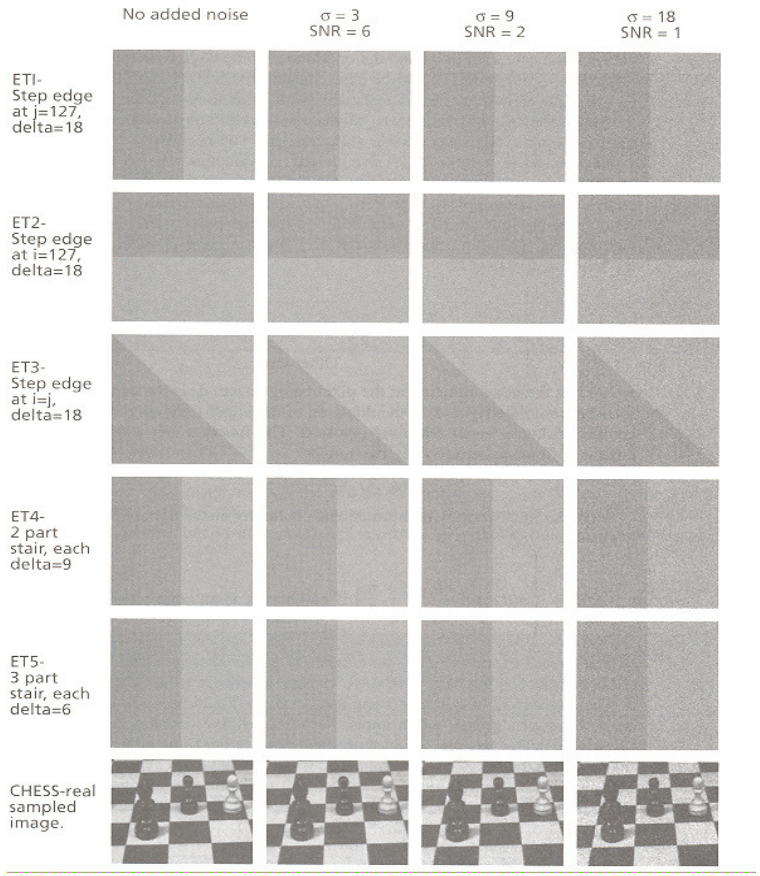


FIG 1 Normal distributed noise and its effect on an image.
(a) Original image (b) Noise having σ=10. (c) Noise having σ=20. (d) Noise having σ=20. (d) Noise having σ=30. (e) Noise having σ=50. (f) Expanded view of an intersection of four regions in the σ=50 image

Test Images

Fig. 2 Standard test images for edge detector evaluation. There are three step edges and two stairs, plus a real sampled image; all have been subjected to normally distributed zero-mean noise with known standard deviations of 3, 9 and 18



Examples

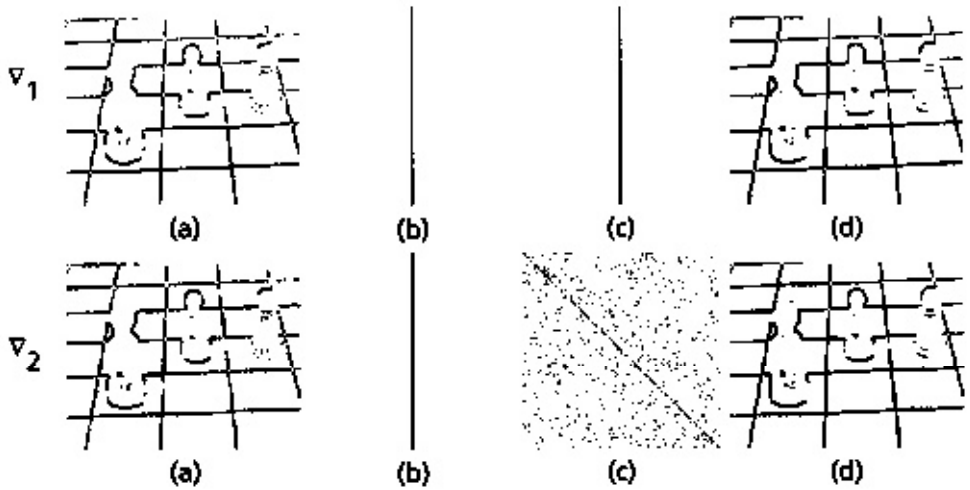


Fig. 3 Sample results from the gradient edge detectors. (a) Chess image (σ=3). (b) Et1 image (SNR=6). (c) ET3 image (SNR=2). (d) Chess image (σ=18).

Measures of Performance of Edge Detection Schemes

- False Positive
- False Negative
- Using Function (Pratt 1978)

$$E_1 = \sum(1/(1+\alpha d(i)^2)) / \max(I_A, I_I)$$

I_A = the number of edge pixels found by the edge detector

I_I= the number of edge pixels in the test image

d(i)=the distance between the actual i*th* pixel and the one found by the edge detector

α is used for scaling

Measures of Performance

- Evaluation scheme based on *local edge coherence* (Kitchen and Rosenfeld)

(a) It measures how well an edge pixel is continued on the left; this function is

$L(k) = a(d,d_k)\ a(k\pi/4, d+\pi/2)$ if neighbor k is an edge pixel

0 otherwise

where d is the edge direction at the pixel being tested

d_0 is the edge direction at its neighbor to the right

d_1 is the direction of the upper-right neighbor, and so on

Counterclockwise about the pixel involved

a= the measure of the angular difference between any two angles

Measures…

$a(\alpha,\beta) = \pi - |\alpha-\beta| / \pi$

(b) A similar function measures directional continuity on the right of the pixel being evaluated:

$R(k) = a(d,d_k)\ a(k\pi/4, d+\pi/2)$ if neighbor k is an edge pixel

0 otherwise

(c) C= Overall continuity measure

= average(L(k), R(k))

(d) Then measure of thinness (T) is applied.

(e) The overall evaluation of the edge detector is:

(f) $E_2 = \gamma C + (1-\gamma)\ T$

The Canny Edge Detector

Canny specified three issues that an edge detector must address. They are:-

1.Error rate:-The edge detector should respond only to edges, and should find all of them; no edges should be missed.

2.Localization:- The distance between the edge pixels as found by the edge detector and the actual edge should be as small as possible.

3.Response:- The edge detector should not identify multiple edge pixels where only a single edge exists.

Canny Edge Detector

- **Canny assumed that a step edge subject to white Gaussian noise.**
- **The edge detector was assumed to be a convolution filter f that would smooth the noise and locate the edge.**
- **So the problem is to identify the one filter that optimizes the three edge detection criteria.**
- **In one –dimension, the response of the filter f to an edge G is given by a convolution integral**

$$H = \int_{W}^{-W} G(-x)f(x)dx$$

Canny…

- **The filter is assumed to be zero outside of the region [-W, W].**
- **Mathematically the three criteria are expressed as:**

$$\text{SNR} = \left(A \left| \int_{-W}^{W} f(x)\,dx \right|\right) / \; n_0 \sqrt{\int_{-W}^{W} f^2(x)\,dx}$$

$$\text{Localization} = A\,|f(0)| / \; n_0 \sqrt{\int_{-W}^{W} f^2(x)\,dx}$$

$$x_{zc} = \pi \left(\int_{-\infty}^{\infty} f^2(x)\,dx\right) / \left(\int_{-\infty}^{\infty} f'^{\,2}(x)\,dx\right)$$

- **The value of SNR is the error rate.**
- **The localization value represents the reciprocal of the distance of the located edge from the true edge.**
- **The value of x_{zc} is the mean distance between zero crossings of f'.**

The Canny Edge Detection Algorithm

Read in the image to be processed I.

- Create a 1D Gaussian mask G to convolve with I. The standard deviation s of this Gaussian is a parameter to the edge detector (call this as G_X or G_y).
- Create a 1D mask for the first derivative of the Gaussian in the x and y directions; call these $G_{x'}$ and $G_{y'}$. The same s value is used as in step 2.

Canny Edge Detection (continued)

- Convolve the image I with G_X along the rows to give the x component image I_X and down the columns (G_Y) to give the y component image I_y .
- Convolve I_X with $G_{X'}$ to give $I_{XX'}$, the x component of I convolved with the derivative of the Gaussian function and
- Convolve I_Y with $G_{Y'}$ to give $I_{YY'}$.
- To view the result the x and y component must be combined. The magnitude of the result at pixel (x,y) is

$$M(x,y) = \sqrt{(I_{XX}'(x,y)^2 + I_{YY}'(x,y)^2)}$$

Canny..

- The final step in the Canny edge detector is a *nonmaximum suppression* step, where pixels that are not local maxima are removed.
- As an extra step, Canny suggests thresholding using hysteresis rather than simply selecting a threshold value to apply everywhere. Hysteresis thresholding uses a high threshold T_h and low threshold T_l.

Examples: Canny

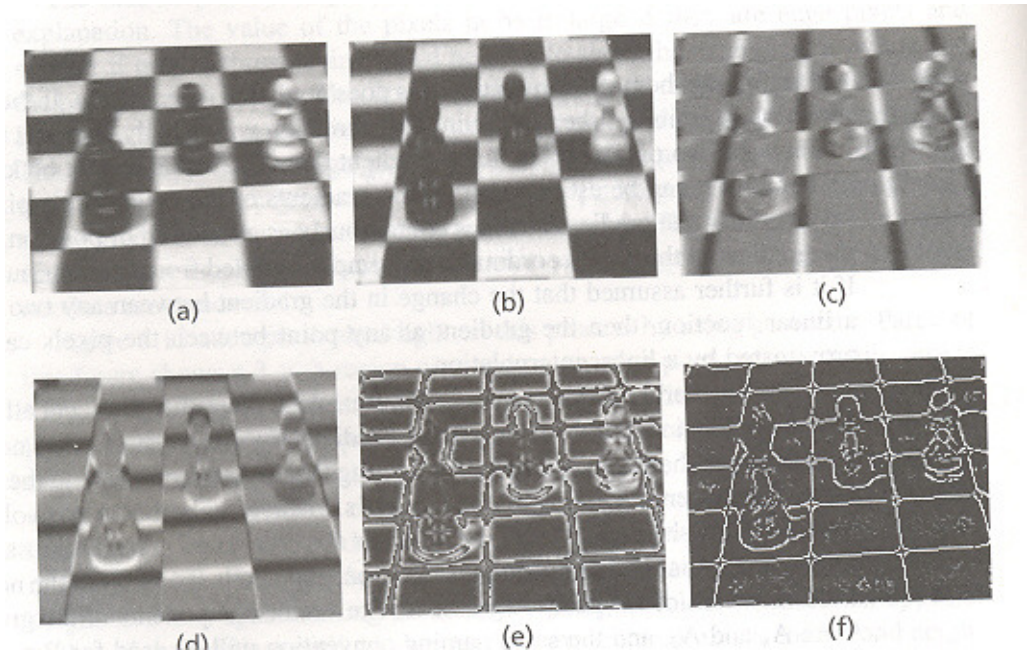


Fig. 7 Intermediate results from the Canny edge detector.
(a) X component of the convolution with a Gaussian. (b) Y component of the convolution with a derivative of a Gaussian. (c) X component of the image convolved with the derivative of a Gaussian (d) Y component of the image convolved with the derivative of a Gaussian. (e) Resulting magnitude image. (f) After nonmaximum suppression.

Examples : Canny

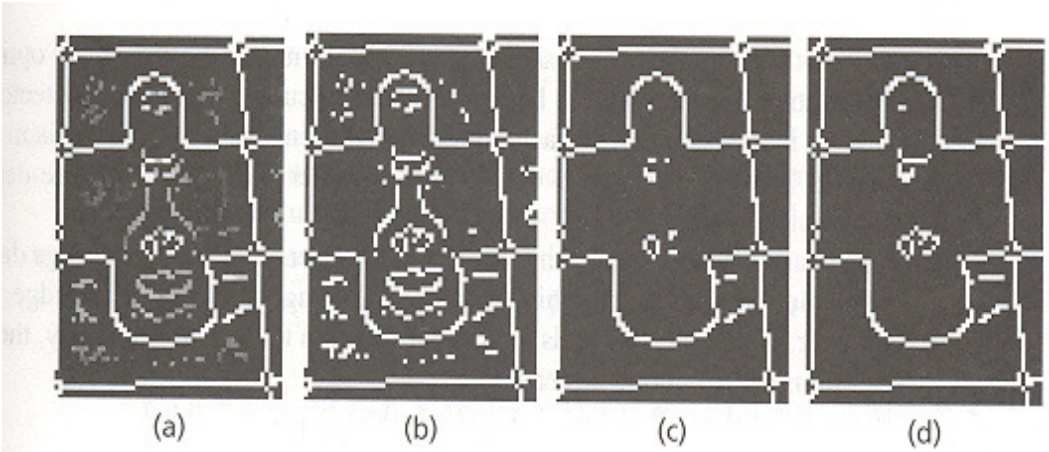
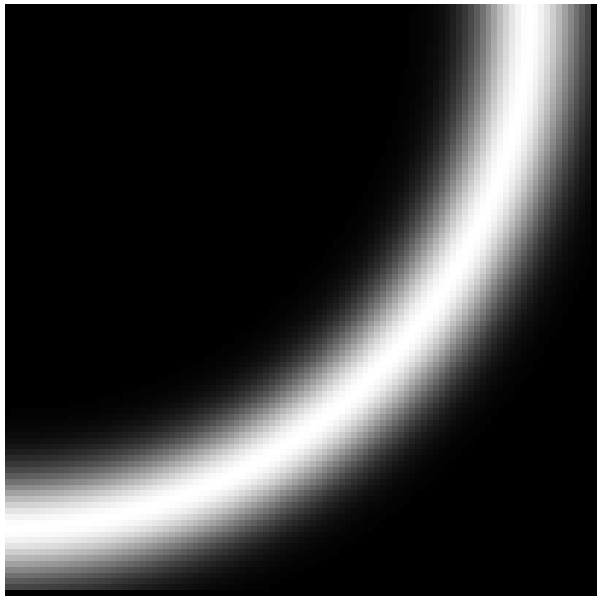
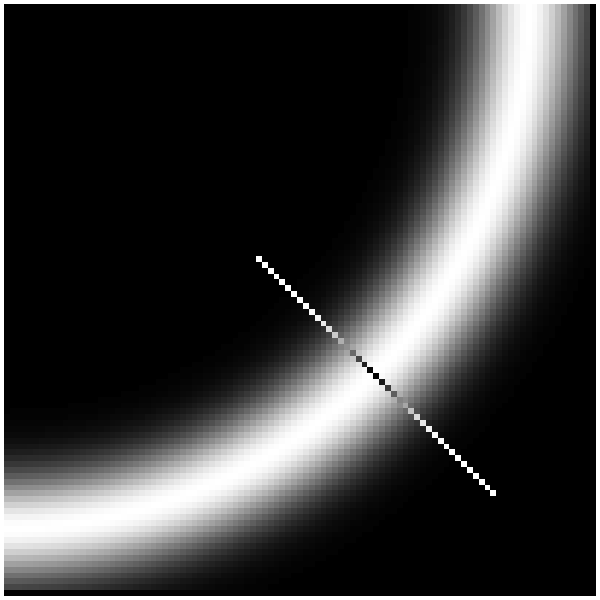


Fig. 8 Hysteresis thresholding. (a) Enlarged portion of figure 7f. (b) This portion after thresholding with a single low threshold. (c) After thresholding with a single high threshold. (d) After hysteresis thresholding.

Edge Detection

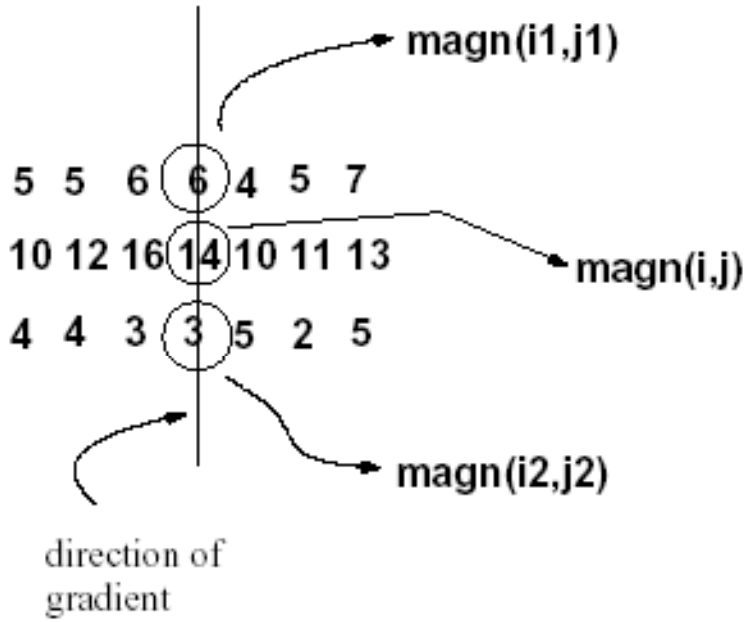
- <u>Non-maxima suppression</u>
  - To find the edge points, we need to find the local maxima of the gradient magnitude.
  - Broad ridges must be thinned so that only the magnitudes at the points of greatest local change remain.
  - All values along the direction of the gradient that are not peak values of a ridge are suppressed.





Edge Detection

- <u>Non-maxima suppression – cont.</u>



Algorithm

For each pixel (x,y) do:

if $magn(i, j) < magn(i_1, j_1)$ or $magn(i, j) < magn(i_2, j_2)$
  then $I_N(i, j) = 0$
else $I_N(i, j) = magn(i, j)$

Edge Detection

- <u>Non-maxima suppression – cont.</u>
  - What are the neighbors?
    - Look along gradient normal
  - Quantization of normal directions:

$$\tan\theta = gradient\ direction$$

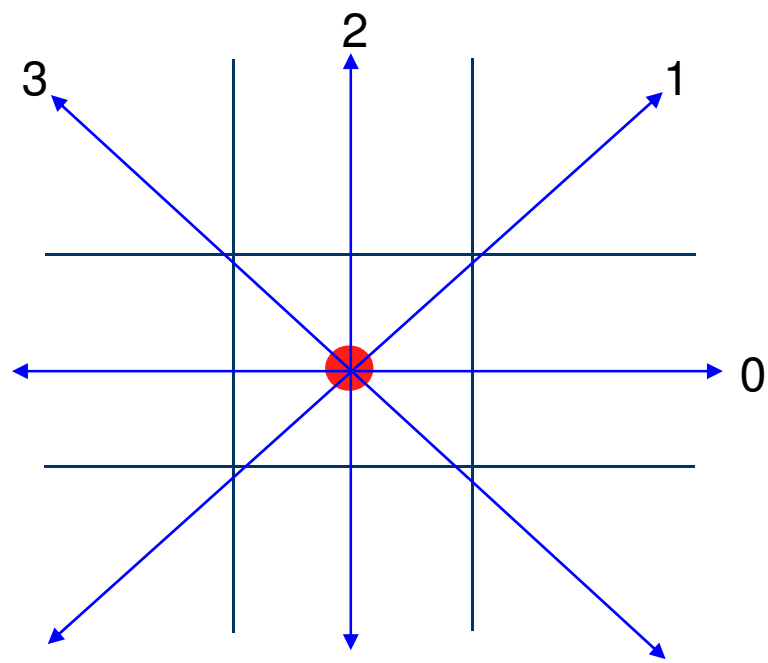
$$\text{Quantization}:$$

$$0: \quad -0.4142 < \tan\theta \le 0.4142$$

$$1: \quad 0.4142 < \tan\theta < 2.4142$$

$$2: \quad |\tan\theta| \ge 2.4142$$

$$3: \quad -2.4142 < \tan\theta \le -0.4142$$



Edge Detection

- Canny – Non-maxima suppression:

gradient magnitude

thinned

Edge Detection

- <u>Hysteresis thresholding / Edge linking</u>
  - The output of non-maxima suppression still contains the local maxima created by noise.
  - Can we get rid of them just by using a single threshold?
    - if we set a low threshold, some noisy maxima will be accepted too.
    - if we set a high threshold, true maxima might be missed (the value of true maxima will fluctuate above and below the threshold, fragmenting the edge).
  - A more effective scheme is to use two thresholds:
    - a low threshold t_l
    - a high threshold t_h
    - usually, $t_h \approx 2t_l$

Edge Detection

- <u>Hysteresis thresholding / Edge linking – cont.</u>

Algorithm

1. Produce two thresholded images $I_1(i, j)$ and $I_2(i, j)$.

(note: since $I_2(i, j)$ was formed with a high threshold, it will contain fewer false edges but there might be gaps in the contours)

2. Link the edges in $I_2(i, j)$ into contours

    2.1 Look in $I_1(i, j)$ when a gap is found.

    2.2 By examining the 8 neighbors in $I_1(i, j)$, gather edge points from $I_1(i, j)$ until the gap has been bridged to an edge in $I_2(i, j)$.

  – The algorithm performs edge linking as a by-product of double-thresholding !!

Edge Detection

- Canny – Hysteresis thresholding / Edge linking

thinned

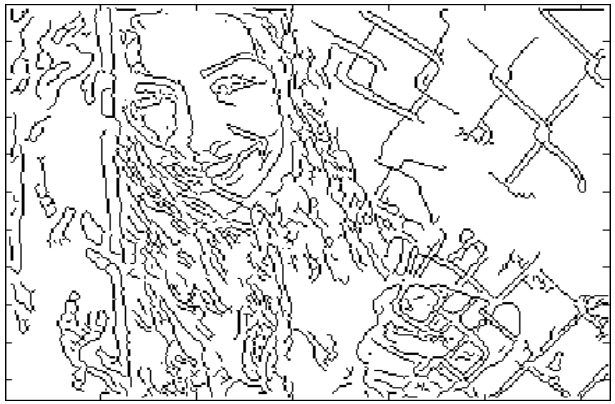
regular

$t = 25$

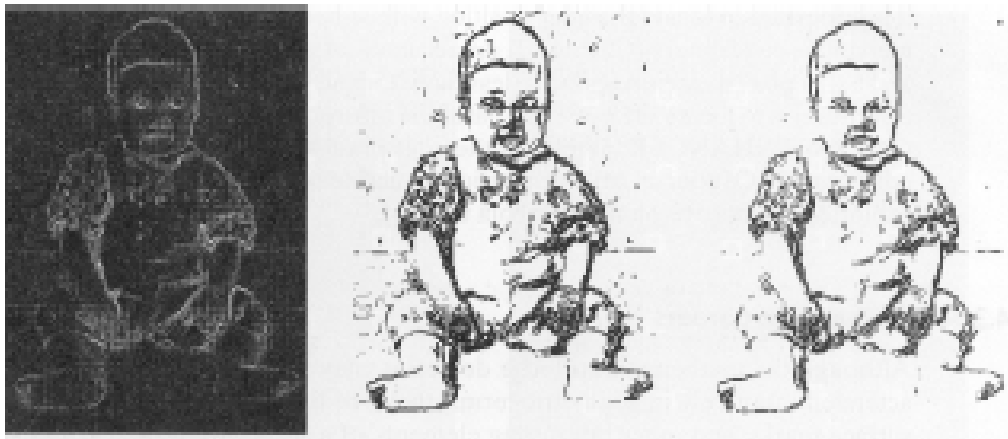
Hysteresis

$High = 35$

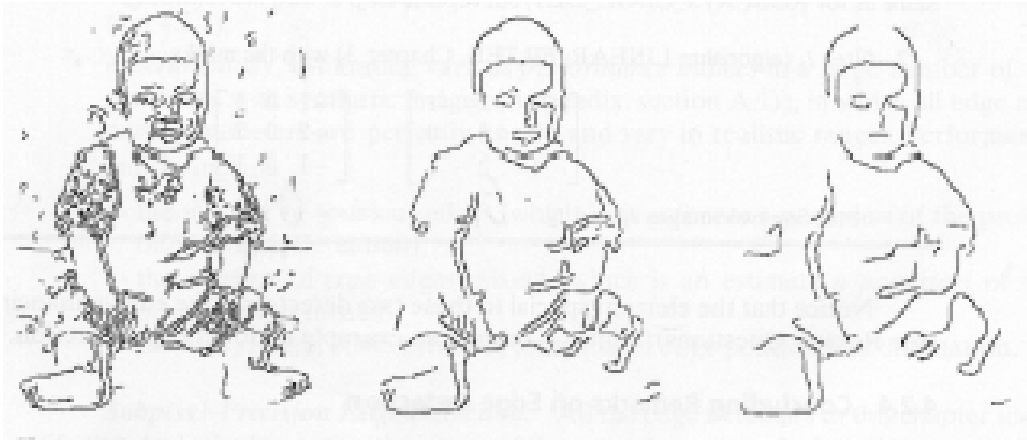
$Low = 15$



Edge Detection

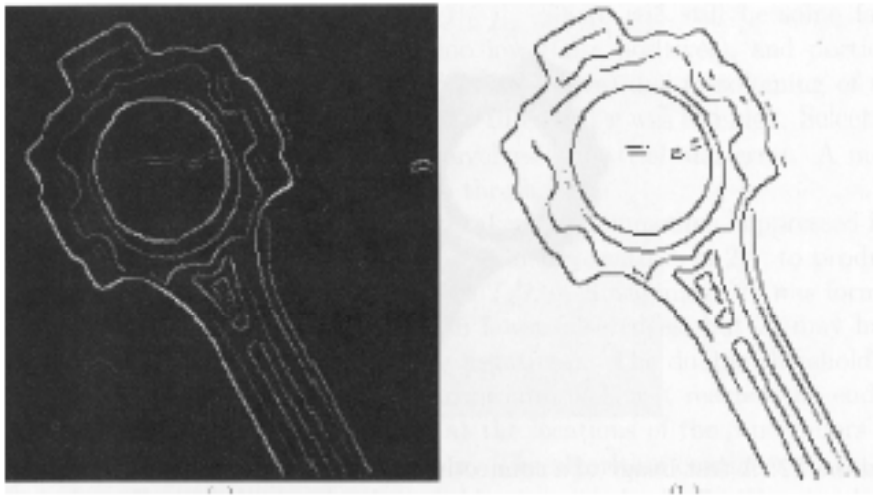


(left:Sobel, middle: thresh=35, right: thersh=50)

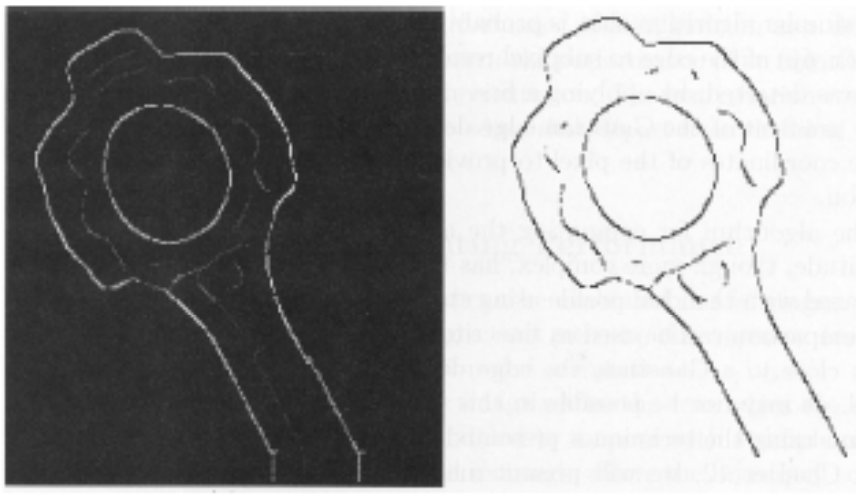


(Canny - left:σ=1, middle: σ=2, right: σ=3)

Edge Detection



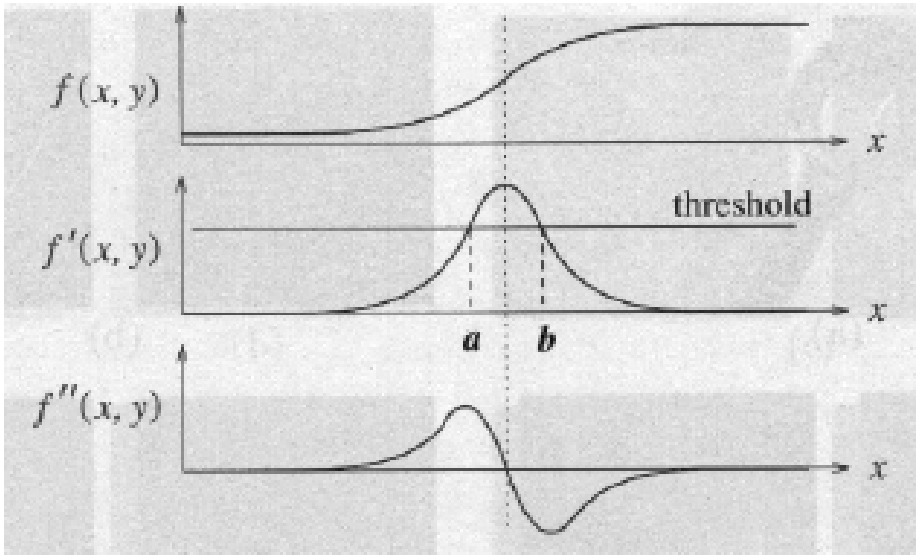
(Canny - 7x7 Gaussian, more details)



(Canny - 31x31 Gaussian, less details)

Edge Detection Using the 2nd Derivative

- Edge points can be detected by finding the zero-crossings of the second derivative.



- There are two operators in 2D that correspond to the second derivative:
  - Laplacian
  - Second directional derivative

Edge Detection Using the 2nd Derivative

- The Laplacian:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Approximating $\nabla^2 f$:

$$\frac{\partial^2 f}{\partial x^2} = f(i, j+1) - 2f(i, j) + f(i, j-1)$$

$$\frac{\partial^2 f}{\partial y^2} = f(i+1, j) - 2f(i, j) + f(i-1, j)$$

$$\nabla^2 f = -4f(i, j) + f(i, j+1) + f(i, j-1) + f(i+1, j) + f(i-1, j)$$

Edge Detection Using the 2nd Derivative

- Example:

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

$$\nabla^2 f = -4z_5 + (z_2 + z_4 + z_6 + z_8)$$

- The Laplacian can be implemented using the mask:

0	1	0
1	-4	1
0	1	0

- Example:

5	5	5	5	5	5
5	5	5	5	5	5
5	5	10	10	10	10
5	5	10	10	10	10
5	5	5	10	10	10
5	5	5	5	10	10

-	-	-	-	-	-
-	0	-5	-5	-5	-
-	-5	10	5	5	-
-	-5	10	0	0	-
-	0	-10	10	0	-
-	-	-	-	-	-

Edge Detection Using the 2nd Derivative

- Properties of the Laplacian
  - It is an isotropic operator.
  - It is cheaper to implement (one mask only).
  - It does not provide information about edge direction.
  - It is more sensitive to noise (differentiates twice).

- Find zero crossings
  - Scan along each row, record an edge point at the location of zero-crossing.
  - Repeat above step along each column

Edge Detection Using the 2nd Derivative

- How do we estimate the edge strength?
- Four cases of zero-crossings :
  - {+,-}
  - {+,0,-}
  - {-,+}
  - {-,0,+}
- Slope of zero-crossing {a, -b} is |a+b|.
- To mark an edge:
  - compute slope of zero-crossing
  - apply a threshold to slope

EDGE MODELS : Marr- Hildreth Edge Detection

Marr studied the literature on mammalian visual systems and summarized these in five major points:-

1. In natural images, features of interest occur at a variety of scales. No single operator can function at all of these scales, so the result of operations at each of scales should be combined.
2. Diffraction patterns does not occur in a natural scene, so some local averaging (smoothing) must take place.

Marr-Hildreth Edge Detection

3. The optimal smoothing filter that matches the observed requirements of biological vision is the Gaussian.
4. When a change in intensity (edge) occurs, there is an extreme value in the first derivative or intensity. This corresponds to a zero crossing in the second derivative.
5. The orientation-independent differential operator of lowest order is the *Laplacian.*

Edge Detection Using the 2nd Derivative

- The Marr-Hildreth edge detector
  - Uses the Laplacian-of-Gaussian (LOG)
  - To reduce the noise effect, the image is first smoothed with a low-pass filter.
  - In the case of the LOG, the low-pass filter is chosen to be a Gaussian.

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

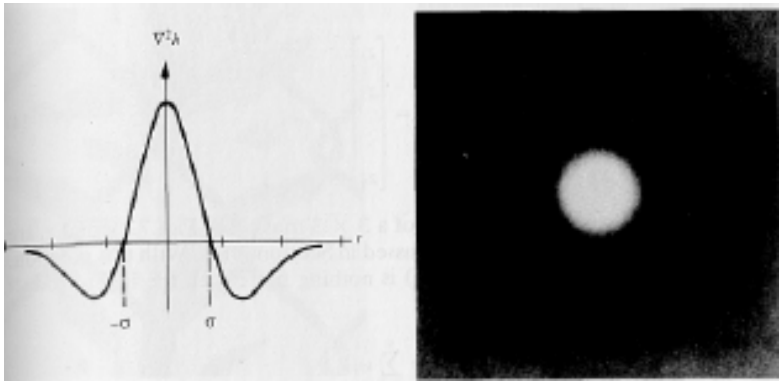
(σ determines the degree of smoothing, mask size increases with σ)

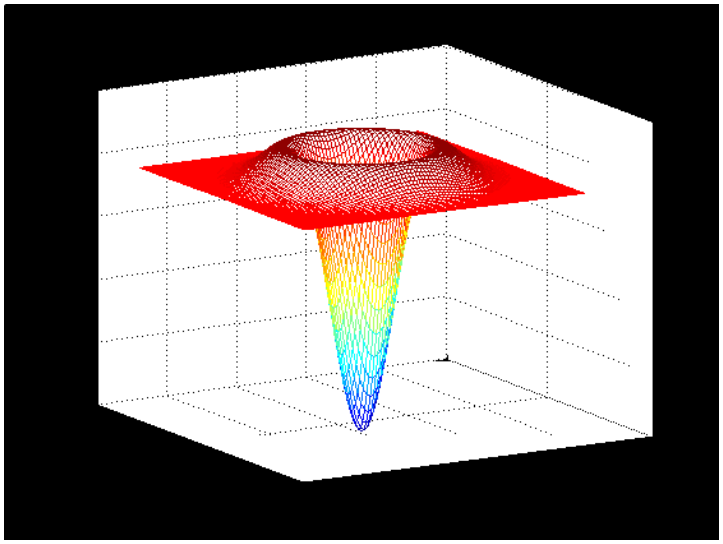
Edge Detection Using the 2nd Derivative

- The Laplacian-of-Gaussian (LOG) – cont.
  - It can be shown that:

$$\nabla^2[f(x,y) * G(x,y)] = \nabla^2 G(x,y) * f(x,y)$$

$$\nabla^2 G(x,y) = \left(\frac{x^2 + y^2 - 2\sigma^2}{\sigma^4}\right) e^{-\frac{x^2+y^2}{2\sigma^2}}$$





Edge Detection Using the 2nd Derivative

- The Laplacian-of-Gaussian (LOG) – cont.

5×5 Laplacian of Gaussian mask

0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

17×17 Laplacian of Gaussian mask

0	0	0	0	0	0	−1	−1	−1	−1	−1	0	0	0	0	0	0
0	0	0	0	−1	−1	−1	−1	−1	−1	−1	−1	−1	0	0	0	0
0	0	−1	−1	−1	−2	−3	−3	−3	−3	−3	−2	−1	−1	−1	0	0
0	0	−1	−1	−2	−3	−3	−3	−3	−3	−3	−3	−2	−1	−1	0	0
0	−1	−1	−2	−3	−3	−3	−2	−3	−2	−3	−3	−3	−2	−1	−1	0
0	−1	−2	−3	−3	−3	0	2	4	2	0	−3	−3	−3	−2	−1	0
−1	−1	−3	−3	−3	0	4	10	12	10	4	0	−3	−3	−3	−1	−1
−1	−1	−3	−3	−2	2	10	18	21	18	10	2	−2	−3	−3	−1	−1
−1	−1	−3	−3	−3	4	12	21	24	21	12	4	−3	−3	−3	−1	−1
−1	−1	−3	−3	−2	2	10	18	21	18	10	2	−2	−3	−3	−1	−1
−1	−1	−3	−3	−3	0	4	10	12	10	4	0	−3	−3	−3	−1	−1
0	−1	−2	−3	−3	−3	0	2	4	2	0	−3	−3	−3	−2	−1	0
0	−1	−1	−2	−3	−3	−3	−2	−3	−2	−3	−3	−3	−2	−1	−1	0
0	0	−1	−1	−2	−3	−3	−3	−3	−3	−3	−3	−2	−1	−1	0	0
0	0	−1	−1	−1	−2	−3	−3	−3	−3	−3	−2	−1	−1	−1	0	0
0	0	0	0	−1	−1	−1	−1	−1	−1	−1	−1	−1	0	0	0	0
0	0	0	0	0	0	−1	−1	−1	−1	−1	0	0	0	0	0	0

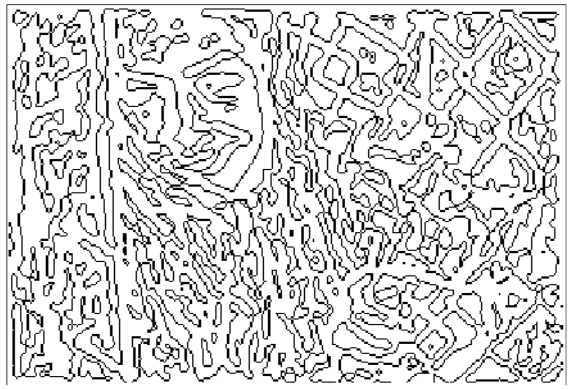
Edge Detection Using the 2nd Derivative

- The Laplacian-of-Gaussian (LOG) – cont.

I

$I * \left(\Delta^2 G\right)$

Zero crossings of $\mathrm{I} * (\Delta^2 G)$



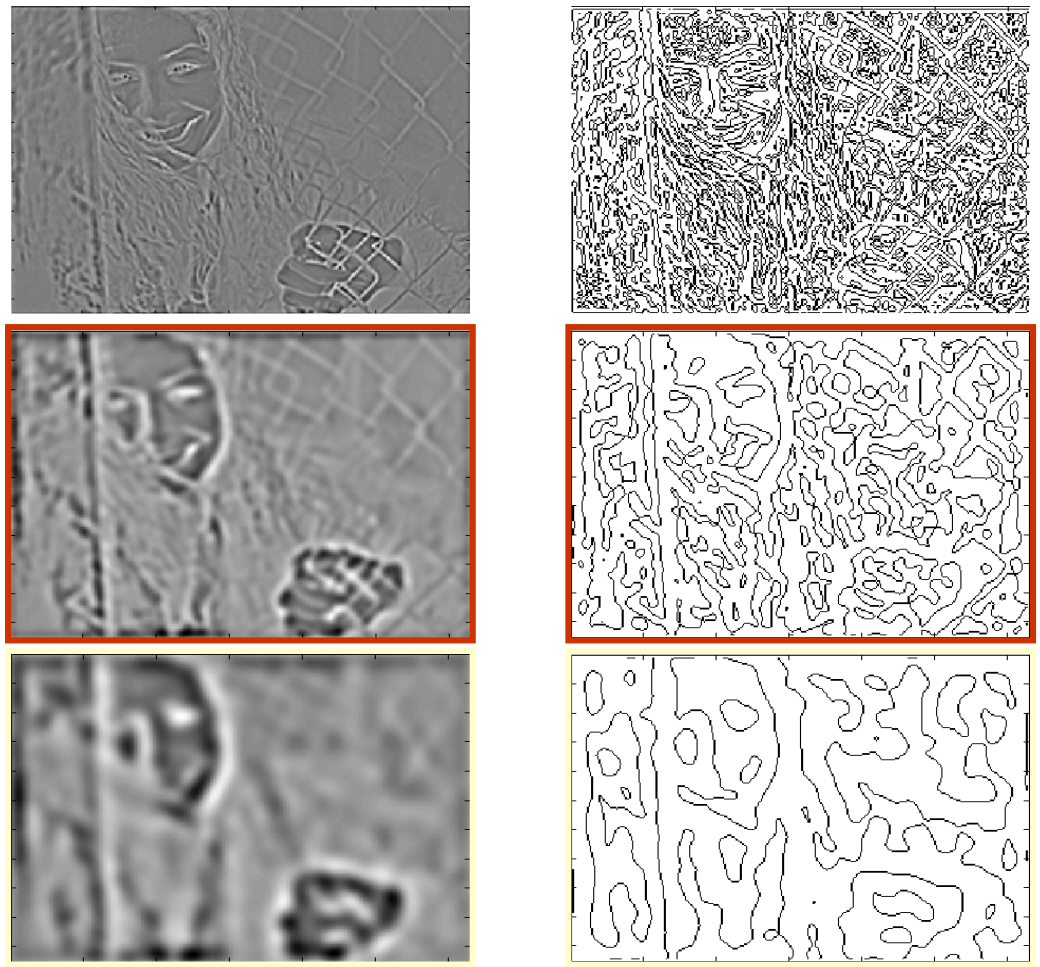
Edge Detection Using the 2nd Derivative

- The Laplacian-of-Gaussian (LOG) – cont.

$\sigma = 1$

$\sigma = 3$

$\sigma = 6$



Edge Detection Using the 2nd Derivative

- Gradient vs. LOG – a comparison
  - Gradient works well when the image contains sharp intensity transitions and low noise
  - Zero-crossings of LOG offer better localization, especially when the edges are not very sharp

2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8

A sample image containing a vertical step edge.

0	0	0	6	−6	0	0	0
0	0	0	6	−6	0	0	0
0	0	0	6	−6	0	0	0
0	0	0	6	−6	0	0	0

2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8

A sample image containing a vertical ramp edge.

0	0	0	3	0	−3	0	0
0	0	0	3	0	−3	0	0
0	0	0	3	0	−3	0	0
0	0	0	3	0	−3	0	0

Edge Detection Using the 2nd Derivative

- Separability
  - Gaussian:
    - A 2-D Gaussian can be separated into two 1-D Gaussians
    - Perform 2 convolutions with 1-D Gaussians

$$h(x,y) = I(x,y) * g(x,y)$$ n^2 multiplications per pixel

$$h(x,y) = \left(I(x,y) * g_1(x)\right) * g_2(y)$$ $2n$ multiplications per pixel

$$g_1 = \begin{bmatrix} .011 & .13 & .6 & 1 & .6 & .13 & .011 \end{bmatrix} \qquad g_2 = \begin{bmatrix} .011 \\ .13 \\ .6 \\ 1 \\ .6 \\ .13 \\ .011 \end{bmatrix}$$

Edge Detection Using the 2nd Derivative

- Separability
  - Laplacian-of-Gaussian:

$$\Delta^2 S = \Delta^2 (g * I) = (\Delta^2 g) * I = I * (\Delta^2 g)$$

Requires n^2 multiplications per pixel

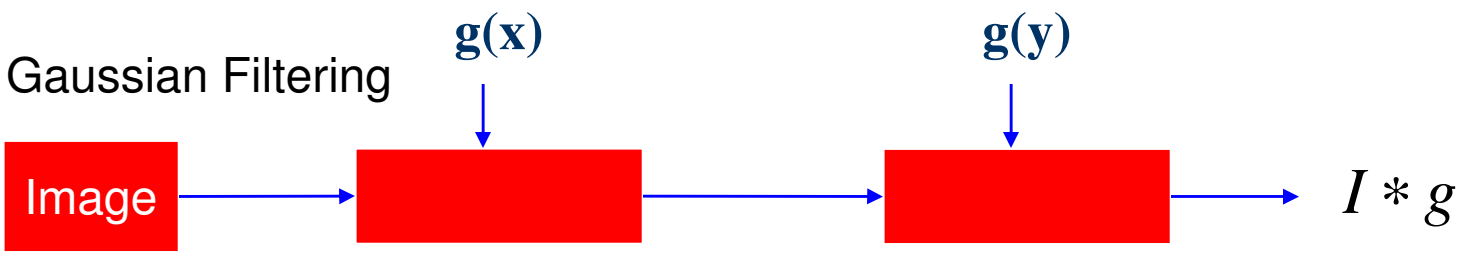
$$\Delta^2 S = (I * g_{xx}(x)) * g(x) + (I * g_{yy}(y)) * g(y)$$

Requires $4n$ multiplications per pixel

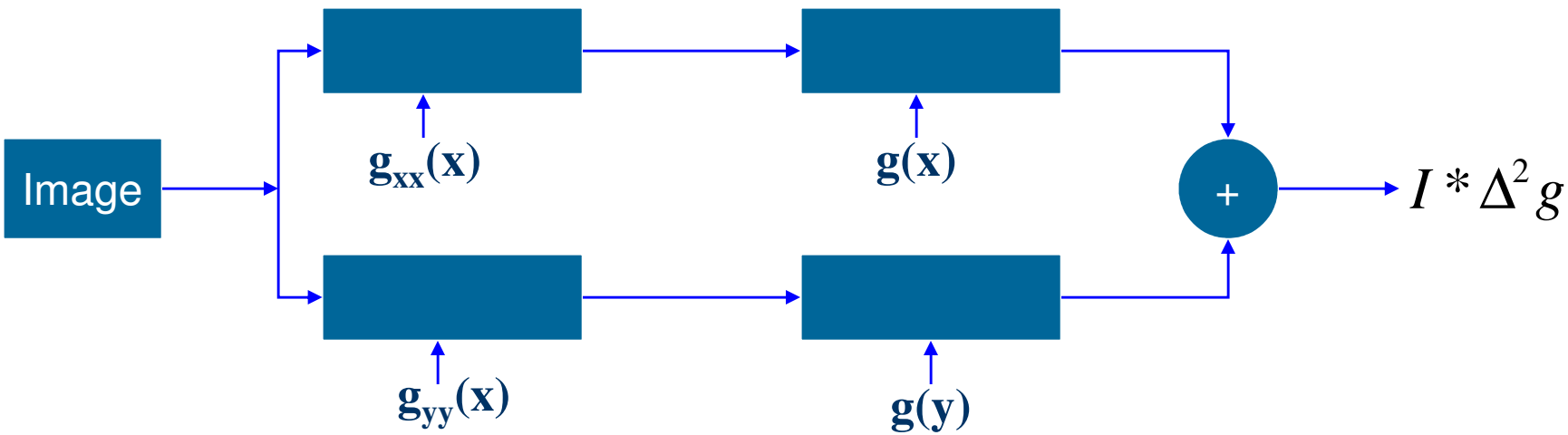
Edge Detection Using the 2nd Derivative

- Separability

Gaussian Filtering



Laplacian-of-Gaussian Filtering



Edge Detection Using the 2nd Derivative

- <u>LOG (Marr-Hildreth) Algorithm:</u>
  - Compute LoG
    - Use one 2D filter: $\Delta^2 g(x, y)$
    - Use four 1D filters: $g(x),\ g_{xx}(x),\ g(y),\ g_{yy}(y)$
  - Find zero-crossings from each row and column
  - Find slope of zero-crossings
  - Apply threshold to slope and mark edges

Marr-Hildreth Edge Detection Algorithm

Summary :

- Convolve the image I with a two- dimensional Gaussian function.
- Compute the Laplacian of the convolved image; call this L
- Edge pixels are those for which there is a zero crossing in L.

Examples

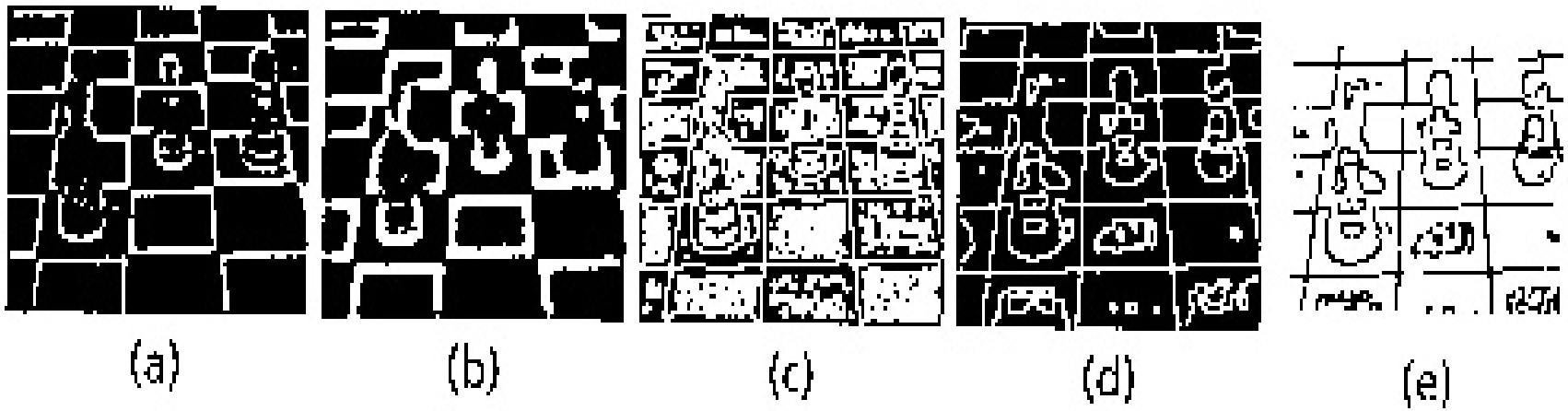


Fig. Steps in the computation of the Marr-Hildreth edge detector. (a) Convolution of the original image with the Laplician of a Gaussian having σ=1.2. (b) Convolution of the image with the Laplician of a Gaussian having σ=2.8. (c) Zero crossings found in (a). (d) Zero crossings found in (b). (e) Result, found by using zero crossings common to both.

Examples

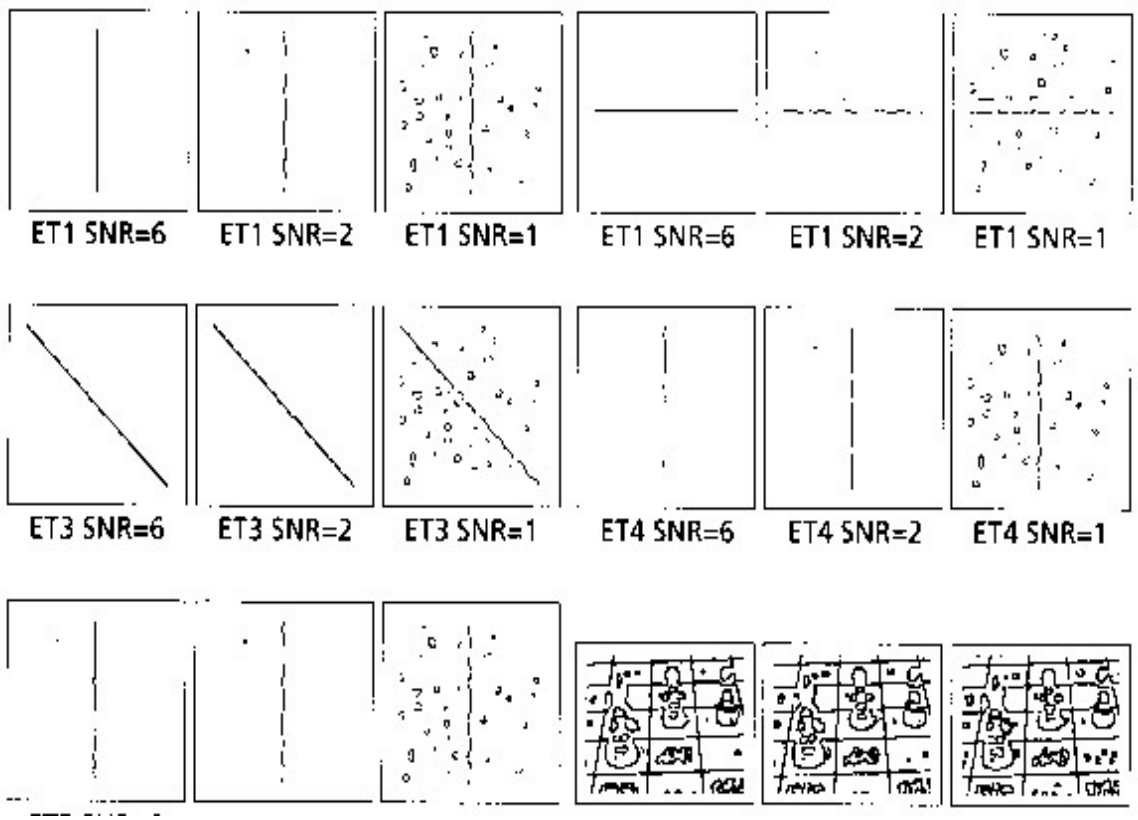


Fig. Edges from the test images as found by the Marr- Hidreth algorithm, using two resolution values

Strength and weaknesses

Advantage:

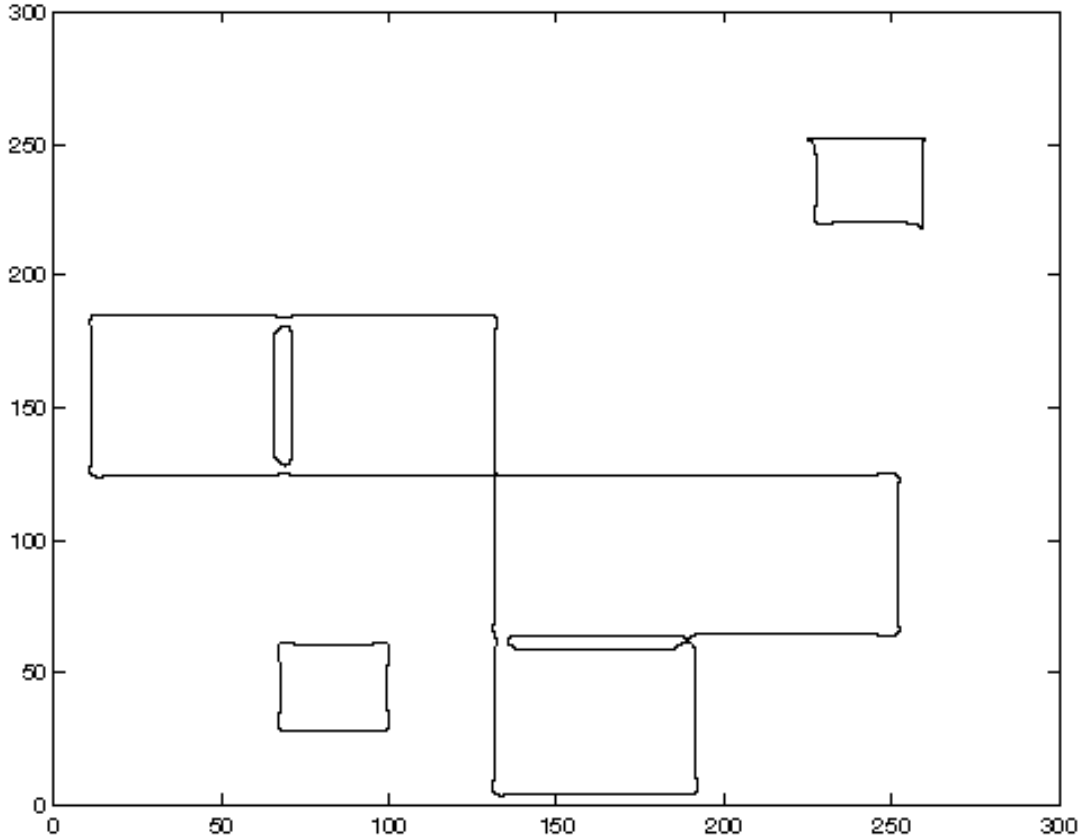
This is much better than the previous ones in cases of low signal to noise ratio.

Disadvantages of the method:-

1) **Low evaluation.**
2) **Locality is not good, the edges are not always thin.**

Weaknesses

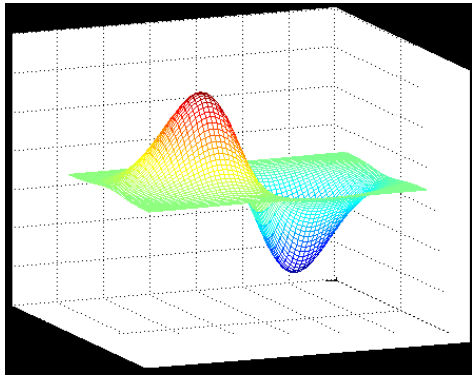
- Disadvantage of LOG edge detection:
  - Does not handle corners well

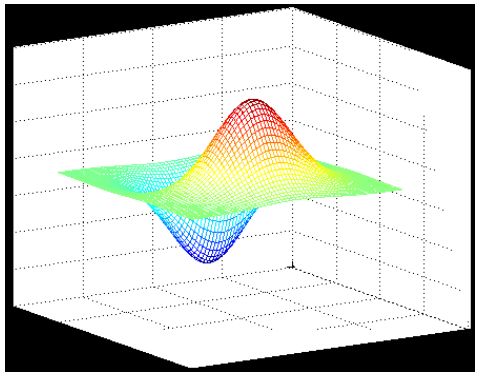


Edge Detection Using the 2nd Derivative

- Disadvantage of LOG edge detection:
  - Does not handle corners well
  - Why?

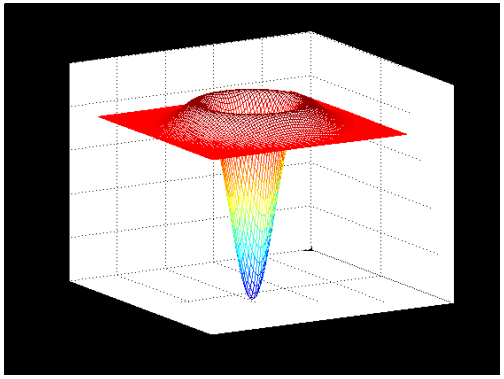
The derivative of the Gaussian:

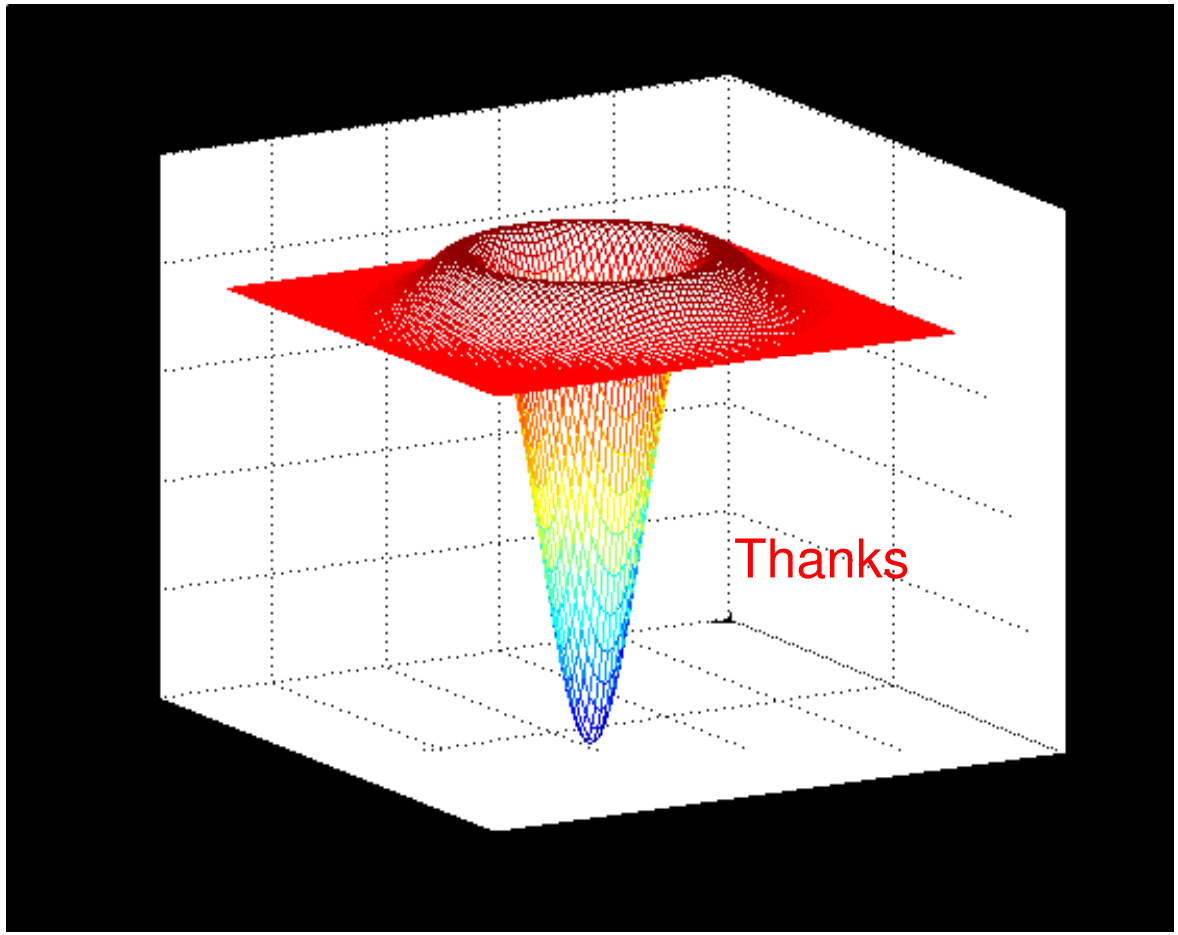




The Laplacian of the Gaussian:

(unoriented)




Thanks